Take the Hint: Speculative Prefill Acceleration for Geo-Distributed Hybrid RAG

Muhammad Abdullah Ali, Muhammad Abdullah Aamir, and Adil Majeed*
Department of Data Science, FAST-NUCES, Islamabad, Pakistan
*adil.majeed@nu.edu.pk

***Abstract*—Deploying disaggregated hybrid Retrieval-Augmented Generation (RAG) across geographic tiers creates an operational bottleneck: while the central GPU idles awaiting remote wide-area-network (WAN) vector search and neural reranking, cold-prefix time-to-first-token (TTFT) stalls in the 2.5–2.8 s range on the three-node tier. We propose SPHP (Speculative Prefill with Hybrid Pipelining), which decouples the query at edge ingress (Node C): it dispatches an ultra-compact 104-byte early hint to the LLM generation host (Node A) while asynchronously initiating dense neural retrieval and fusion on the retrieval host (Node B). SPHP overlaps GPU prompt prefill with WAN transport and dense neural ranking while maintaining a bounded KV-cache reserve, committing only after a set-overlap verification barrier. On our 3-node physical testbed under realistic WAN latencies (15–80 ms) and packet loss (0–2%), SPHP reduces end-to-end cold TTFT by 16.9%–26.7% (up to 34.5% under peak asymmetric loss), keeps warm inter-tier sensitivity at or below 0.153 ms/ms, and bounds answer drift to lexical rephrasing rather than factual degradation.**

***Index Terms*—Retrieval-Augmented Generation, Edge–Cloud Systems, Speculative Inference, Geo-Distributed Systems, Placement Optimization.**

I. Introduction

Retrieval-Augmented Generation (RAG) grounds large language models (LLMs) in external factual corpora, mitigating hallucinations and expanding parametric knowledge [1]–[4]. Modern production pipelines increasingly adopt *hybrid* architectures combining lexical sparse retrieval (BM25 [5]) for exact keyword matching, dense neural representations (BGE-M3 [6], DPR [7]) for semantic generalization, reciprocal-rank fusion (RRF [8]), passage hydration, and autoregressive generation under optimized serving engines like vLLM [9].

Deploying hybrid RAG across geo-distributed edge–cloud fleets exposes a fundamental *placement dilemma*. Constituent pipeline stages exhibit radically divergent computational and memory footprints. Dense embedding computation and LLM generation require specialized GPU accelerators with substantial VRAM, whereas sparse inverted-index traversal is memory-bandwidth- and CPU-bound, and document hydration is an $O(1)$ key-value lookup. Although deep neural network (DNN) layer partitioning [10]–[12] and LLM phase disaggregation (prefill vs. decode) [13]–[17] have been investigated for monolithic models, the systems principles governing multi-stage hybrid RAG pipelines partitioned across heterogeneous, wide-area topologies remain uncharacterized.

Through physical multi-tier testbed measurements over the 8.84M-passage MS MARCO benchmark [18], we uncover two governing systems properties:

1) *Decoupling of WAN Delay from Payload Bulk*: When wide-area boundaries transmit compact document identifiers (104–144 bytes) rather than hydrated text (≈6 KB), time-to-first-token (TTFT) increases by only ≈0.95 ms per millisecond of round-trip time (RTT). The network transit delay is effectively linear and predictable.
2) *KV-Cache State Dominates Latency Variance*: In contrast to network delay, KV-cache state dominates TTFT variance. Cold-prefix TTFT reaches 2.8–3.2× warm steady-state latency (1,656–1,720 ms cold vs. 510–588 ms warm), largely driven by transformer prefill and GPU page-table allocation. Serializing dense neural retrieval before GPU prefill leaves generation GPUs idle while awaiting candidate sets.

To exploit this retrieval-prefill asymmetry, we introduce **SPHP** (Speculative Prefill with Hybrid Pipelining). Upon completing BM25 search on the edge tier (Node C), SPHP streams an early provisional sparse hint ($\mathcal{H}$, 104 bytes) directly to the generation host (Node A), initiating speculative prefill in GPU VRAM concurrently with in-flight dense neural embedding and fusion on Node B. When the verified ranking ($\mathcal{F}$, 144 bytes) arrives, Node A evaluates a formal set-overlap verification barrier (Overlap $\geq \theta$). On a hit ($h = 0.900$), the resident KV-cache commits without stall, cutting cold TTFT by 16.9%–26.7% on the physical three-node testbed (up to 34.5% under peak asymmetric loss) without factual degradation.

This paper makes four primary contributions:

- *Empirical Critical-Path Characterization*: We provide the first physical multi-tier characterization of geo-distributed hybrid RAG, quantifying the retrieval-prefill asymmetry and the dominance of KV-cache state over wide-area transit delay.
- *SPHP Protocol*: We design and implement SPHP, an asynchronous speculative pipeline protocol that overlaps GPU prefill with remote neural retrieval across wide-area boundaries.
- *Analytical Placement Cost Model*: We formulate and cross-validate a closed-form cost model (leave-one-tier-out MAPE 2.15% cold / 4.13% warm), demonstrating that speculative split placement remains optimal across the evaluated wide-area regimes.
- *Physical Testbed Evaluation & Robustness*: Across physical three-node tiering (CM-2), unshaped public internet routes (CM-4), a 27-cell asymmetric loss matrix (CM-5), and automated edge failovers, SPHP maintains consistent acceleration

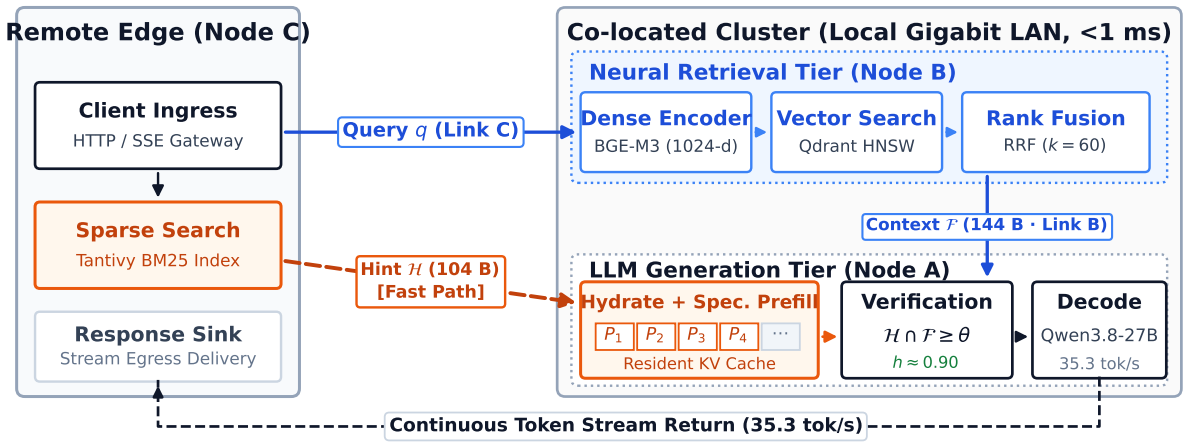


Fig. 1. SPHP architecture and speculative pipelining across distributed tiers. Edge node C dispatches an early hint $\mathcal{H}$ (104 B) to overlap GPU prefill on generation node A with neural retrieval on retrieval node B. Nodes A and B are co-located on a local LAN; Link B is netem-swept (0–80 ms) to characterize multi-tier disaggregation.

and high speculation hit rates (81.3%–86.9%).

The remainder of this paper is organized as follows: Section II reviews related work in hybrid RAG and disaggregated inference. Section III presents our empirical characterization of the distributed critical path. Section IV introduces the SPHP protocol and analytical cost model. Section V evaluates the system on our physical testbed under diverse network conditions. Section VI discusses systems implications and concludes the paper.

II. Background and Related Work

Retrieval-augmented generation bridges parametric memorization and external dynamic knowledge [1]–[4]. Hybrid retrieval pairs exact keyword precision from inverted indices (BM25 [5]) with semantic dense vector search (DPR [7], BGE-M3 [6]) through reciprocal-rank fusion (RRF [8]). Shen *et al.* [19] analyzed systems trade-offs in monolithic RAG serving on single instances, demonstrating that retrieval latency can dominate execution. However, monolithic studies do not consider network cuts, geo-distribution, or disaggregated multi-tier stage placement.

Speculative decoding accelerates autoregressive generation by drafting candidate tokens on a smaller auxiliary model and verifying them in parallel on the target LLM [20], [21]. In distributed settings, *Speculation at a Distance* [22] showed that wide-area round-trip times severely undermine token-level speculation due to round-trip latency amplification. SPHP operates on an orthogonal systems dimension: rather than speculating over output tokens across the network, SPHP speculates over the *pipeline schedule* on the generation host. While Speculative RAG [23] and Omnia [24] explore speculative drafting across document subsets without pipeline reconciliation, SPHP leverages an early provisional sparse hint to overlap prefill with dense retrieval, using set overlap as a formal verification barrier.

Partitioning neural network execution between edge and cloud has been studied for DNN layer partitioning (Neurosurgeon [10], Auto-Split [11], DDNN [12]) and LLM phase disaggregation (Splitwise [13], DistServe [14], Janus [15], Mooncake [16], Helix [17]). These systems separate prefill and decode instances over datacenter interconnects. In contrast, our work addresses multi-stage hybrid RAG pipelines where constituent stages (sparse traversal, dense neural encoding, rank fusion, document hydration, autoregressive decoding) possess radically divergent compute, memory, and storage footprints across wide-area network boundaries.

III. Empirical Characterization of Disaggregated RAG

A. Physical Testbed Architecture

Our physical testbed couples three heterogeneous machines across wide-area networks over WireGuard tunnels [25] (Figure 1):

- *Node C (Edge Ingress)*: Hosted on a 12th-generation Intel Core laptop (16 GB RAM) situated at a remote edge site connected over a wide-area link (Link C, 15 ms RTT). Node C terminates client HTTP connections, parses queries, executes Tantivy BM25 sparse lexical search over the 8.84M-passage MS MARCO index [18], and streams response tokens back to users via Server-Sent Events (SSE).
- *Node B (Neural Retrieval)*: Hosted on an Intel Core i7-10750H laptop equipped with an NVIDIA GTX 1660 Ti GPU (6 GB VRAM) and 24 GB RAM. Node B computes 1024-dimensional BGE-M3 dense embeddings [6], queries an HNSW vector index in Qdrant, and executes reciprocal-rank fusion ($k = 60$) [8].
- *Node A (LLM Generation)*: Powered by an AMD Ryzen 7 7700 processor and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM). Node A serves the Qwen3.8-27B generator [26] under vLLM [9] with AutoRound W4A16 quantization [27]. Node A maintains an indexed SQLite passage store for $O(1)$ hydration, evaluates candidate overlap at the verification barrier, and executes prefill and decoding.

Nodes A and B are physically co-located in the same room on a local Gigabit LAN (< 1 ms physical RTT). To evaluate disaggregated topologies, Link B is shaped via Linux `netem`

TABLE I
PLACEMENT TOPOLOGY TAXONOMY (P0–P3). PIPELINE STAGES PARTITIONED ACROSS REMOTE EDGE INGRESS (NODE C), RETRIEVAL HOST (NODE B), AND GENERATION HOST (NODE A).

Topol.	Paradigm	Edge (Node C)	Retrieval (Node B)	Generation (Node A)
P0	Colocated	Ingress	—	All stages
P1	Cloud Baseline	Ingress proxy	—	Sparse, Dense, Fusion, Hydrate, Gen
P2	3-Node Split	Ingress, Sparse	Dense, Fusion	Hydrate, Prefill, Decode
P2-SPHP	Spec. 3-Node	Ingress, Sparse, Hint	Dense, Fusion	Hydrate, Spec. Prefill, Decode
P2-Consol	2-Node Control	Ingress	Sparse, Dense, Fusion	Hydrate, Prefill, Decode
P3	Edge Hydrate	Ingress, Sparse	Dense, Fusion, Hydrate	Prefill, Decode

[28] across four delay tiers: 0 ms (LAN control), 15 ms (metro), 40 ms (regional), and 80 ms (transcontinental) at 100 Mbps, verified before and after each run against a $\pm 30\%$ ping gate and ± 0.05 pp loss gate. The wide-area boundary transmits only compact records: a 104-byte provisional hint (top-5 IDs), a 144-byte verified ranking (top-10 IDs), or 6,064-byte hydrated text, with Node A's SQLite store providing $O(1)$ passage lookup so that no full-document payload ever traverses the WAN on the SPHP path.

We define candidate placement topologies based on where wide-area boundaries bisect the pipeline (Table I). P0 colocates all stages on the generation host, whereas P1 routes ingress through Node C but executes all retrieval and generation on Node A. In the primary physical three-node topology (P2), lightweight sparse search executes on Node C, while memory-intensive dense retrieval executes on Node B. To minimize WAN bandwidth, only top-k document identifiers (104–144 bytes) cross the network to Node A for hydration and generation. The speculative protocol (P2-SPHP) enables Node C to dispatch an early provisional sparse hint (104 bytes for top-5 IDs) directly to Node A over Link C, initiating masked prefill in GPU VRAM concurrently with neural retrieval on Node B. When Node C and Node B are co-located at the same site, the topology collapses to the consolidated two-node control baseline (P2-Consol), which we use to isolate internal stage mechanics and parameterize our analytical cost model. Finally, P3 evaluates shifting document hydration to the edge, transmitting hydrated raw text ($\approx$6 KB) across the WAN.

B. Critical-Path Latency Breakdown

Table II details per-stage latencies across cache states and network tiers. In the warm steady-state, sparse lexical search completes in 68–89 ms, whereas dense neural search requires 111–118 ms, exhibiting a consistent 1.3–1.6$\times$ asymmetry. In the cold-prefix state, sparse search requires 576–640 ms (dominated by OS page faults across the 8.84M inverted index), while dense encoding and index traversal require 1,029–1,130 ms ($\approx$1.8$\times$). Crucially, the parallel retrieval barrier ($\max(T_{\text{sparse}}, T_{\text{dense}})$) consumes only 111–118 ms of the warm TTFT budget (510–588 ms). The *prefill residual*—encompassing passage hydration, tokenization, transformer

TABLE II
BASELINE PER-STAGE LATENCY AND TTFT ON THE TWO-NODE CONSOLIDATED CONTROL BY NETWORK REGIME AND CACHE STATE (120 QUERIES/CELL, 3 REPEATS). TPS IS MEDIAN AUTOREGRESSIVE DECODE THROUGHPUT.

		Sprs	Dnse	TTFT (ms)			Total	TPS
RTT	State	(ms)	(ms)	mean	p50	p95	(ms)	(tok/s)
0 ms	cold	575.5	1104.4	1656.8	1637.2	2183.6	2396.7	35.2
0 ms	warm	68.4	111.1	510.0	470.6	782.9	1592.6	35.2
15 ms	cold	596.1	1103.6	1663.8	1656.0	2002.3	2443.4	35.4
15 ms	warm	78.3	112.6	531.7	500.3	785.0	1701.1	35.3
40 ms	cold	640.3	1129.7	1720.0	1697.1	2258.4	2590.5	35.4
40 ms	warm	88.8	118.2	560.9	538.7	833.1	1826.6	35.3
80 ms	cold	600.7	1029.0	1640.5	1633.3	2082.8	2224.8	35.4
80 ms	warm	81.3	113.9	587.6	554.9	832.3	1445.3	35.5

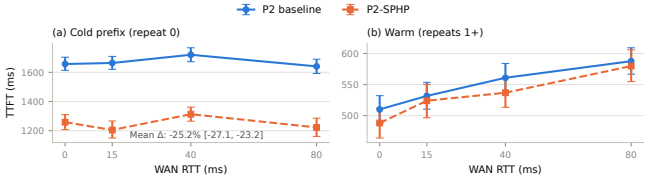


Fig. 2. Measured TTFT vs. WAN RTT across network regimes (error bars denote bootstrap 95% confidence intervals). (a) Cold-prefix state: SPHP consistently accelerates TTFT by 23.7%–27.6%. (b) Warm steady-state: curves exhibit linear RTT slope ($\approx$0.95 ms/ms), confirming payload decoupling.

context prefill, and engine queueing—demands 392–464 ms in the warm state. Consequently, optimizing network transit cannot yield substantial savings; latency acceleration must target overlapping the prefill residual.

The latency disparity between cache states in Table II overshadows network variations. Cold TTFT (1,656.8–1,720.0 ms) is 2.8–3.2$\times$ warm TTFT (510.0–587.6 ms) across all evaluated tiers. Across the full 0 to 80 ms RTT sweep, warm TTFT increases by only 77.6 ms (slope $\approx$0.95 ms/ms), precisely reflecting a single WAN crossing for identifier payloads, while cold TTFT remains statistically flat across network tiers (Figure 2).

Autoregressive decode throughput (R_{decode}) is calculated as:

$$R_{\text{decode}} = \frac{N-1}{T_{\text{decode}}} = \frac{N-1}{10^{-3}\,(t_{\text{end}} - t_{\text{first_token}})}, \qquad (1)$$

where N is emitted tokens and timestamps are in milliseconds, $t_{\text{first_token}}$ is client arrival of the initial streaming token (marking TTFT completion), and t_{end} is stream termination. Equation (1) strictly isolates memory-bandwidth-bound autoregressive decoding from prompt prefill. As shown in Table II, R_{decode} remains constant at 35.2–35.5 tok/s across all network regimes, confirming that wide-area transit bottlenecks only the delivery of the initial token; once token emission commences, delivery proceeds at line rate without streaming stalls.

IV. THE SPHP PROTOCOL AND COST MODEL

A. Protocol Mechanics

SPHP pipelines retrieval and generation across the three-tier hierarchy (Figure 3). Rather than serializing dense retrieval and fusion prior to execution on Node A, SPHP exploits the temporal head start of edge sparse traversal through an asynchronous protocol. Upon completing BM25 search at

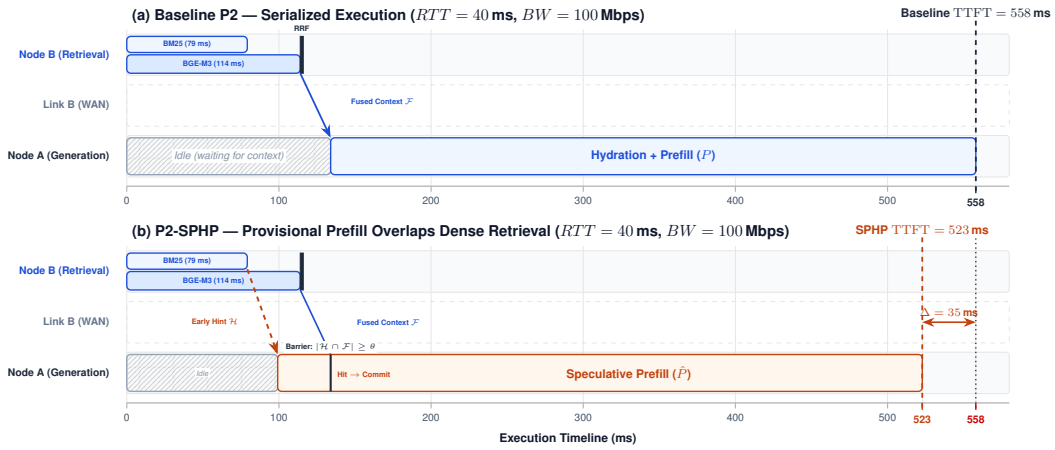


Fig. 3. Space-time execution swimlanes for baseline P2 vs. P2-SPHP at 40 ms WAN RTT (100 Mbps, warm steady-state). The baseline serializes retrieval and WAN transit, idling the generation GPU for 134 ms. SPHP overlaps 424 ms of masked transformer prefill on Node A with in-flight dense retrieval on Node B, cutting warm TTFT from 558 to 523 ms.

time T_{sparse}, Node C immediately streams a provisional hint consisting of top-5 sparse passage identifiers ($\mathcal{H}$, 104 bytes) over Link C to Node A, while BGE-M3 dense encoding and HNSW search proceed concurrently on Node B over the full query payload forwarded at ingress.

While dense retrieval is in flight on Node B, Node A hydrates provisional passages from its local SQLite store and initiates speculative masked prefill $\hat{P}$ in VRAM. Candidate tokens generated during this window remain buffered internally and are strictly masked from the client SSE stream, so no unverified content reaches the user. Across evaluated regimes, SPHP reduces cold TTFT by 23.7%–27.6% (-24.1% at 0 ms, -27.6% at 15 ms, -23.7% at 40 ms, and -25.5% at 80 ms RTT). All four one-sided permutation tests clear the Holm-FWER significance threshold ($p \leq 0.0008$, Cohen's $d = -1.29$ to -1.57); in absolute terms at 0 ms RTT, mean cold TTFT drops from 1,656.8 ms to 1,258.4 ms (-398.4 ms).

When the verified ranking ($\mathcal{F}$, 144 bytes for top-10 IDs) arrives from Node B at time $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}$, Node A compares the provisional hint against the top-k_h fused prefix $\mathcal{F}_{1:k_h}$ ($k_h = 5$):

$$\text{Overlap}(\mathcal{H}, \mathcal{F}) = \frac{|\mathcal{H} \cap \mathcal{F}_{1:k_h}|}{|\mathcal{H}|}. \tag{2}$$

If $\text{Overlap}(\mathcal{H}, \mathcal{F}) \geq \theta$ with $\theta = 0.5$, speculation hits: $\hat{P}$ is committed, the provisional KV-cache is preserved, and autoregressive decoding commences without stall. On a speculation miss, $\hat{P}$ is discarded, paying an abort penalty T_{miss} (124.5 ms warm, 1,076.6 ms cold), and full prefill executes over the verified ranking $\mathcal{F}$. Dense retrieval thus serves as an *asynchronous semantic guardrail*: on queries where lexical matching diverges from semantic relevance, the candidate sets disagree, triggering an automatic abort and preserving full hybrid recall guarantees. Across the benchmark, the empirical speculation hit rate reaches $h = 0.900$ (95% CI $[0.883, 0.914]$) with a mean candidate set overlap of 0.636.

Because committed hit-path responses are generated from provisional sparse context, we evaluate answer drift across 200 seeded dev queries against baseline P2. Re-generating responses from identical fused contexts under baseline P2 establishes an empirical upper bound of token-F1 $= 0.586$ (Exact Match 0.121) due to decode sampling stochasticity. Hit-path SPHP responses achieve a token-F1 agreement of 0.422 (Exact Match 0.042) against baseline. Qualitative inspection confirms that divergence stems from minor lexical rephrasings rather than factual degradation.

B. Analytical Cost Model

We formulate an analytical cost model predicting TTFT as a function of payload size p, round-trip time rtt, and link bandwidth bw (Mbps):

$$T_{\text{WAN}}(p, \text{rtt}, \text{bw}) = \frac{\text{rtt}}{2} + \frac{8\,p}{\text{bw} \times 10^6} \times 10^3 \text{ ms}. \tag{3}$$

Payload sizes are $p_{\text{id}} = 144$ B (top-10 IDs), $p_{\text{hint}} = 104$ B (top-5 hint), and $p_{\text{text}} = 6{,}064$ B (hydrated text). Letting $T_{\text{ret}} = \max(T_{\text{sparse}}, T_{\text{dense}})$, the expected TTFT equations for non-speculative baselines are:

$$T^{(0)}_{\text{TTFT}} = T_{\text{ret}} + T_{\text{fusion}} + T_{\text{hydrate}} + T_{\text{prefill}}, \tag{4}$$

$$T^{(2)}_{\text{TTFT}} = T^{(0)}_{\text{TTFT}} + T_{\text{WAN}}(p_{\text{id}}), \tag{5}$$

$$T^{(3)}_{\text{TTFT}} = T^{(0)}_{\text{TTFT}} + T_{\text{WAN}}(p_{\text{text}}), \tag{6}$$

while the speculative SPHP placement cost is modeled as:

$$\begin{aligned} T^{(2,\text{SPHP})}_{\text{TTFT}} &= h\big(\max(t_f, t_p) - C_{\text{c}}\big) \\ &\quad + (1-h)(T^{(2)}_{\text{TTFT}} + T_{\text{miss}}), \end{aligned} \tag{7}$$

where $t_s = T_{\text{sparse}} + T_{\text{WAN}}(p_{\text{hint}}) + T_{\text{hydrate}}$, $t_p = t_s + T_{\text{prefill}}$, and $t_f = T_{\text{dense}} + T_{\text{fusion}} + T_{\text{WAN}}(p_{\text{id}})$. The term C_{c} (64 ms cold, 0 warm) represents the *overlapped memory allocation credit*: on cold engines, launching provisional prefill absorbs GPU page-table allocation and CUDA graph instantiation during the retrieval head-start window ($\Delta t = T_{\text{dense}} - T_{\text{sparse}} \approx 489$ ms), which baseline P2 serializes on the critical path.

Stage latency parameters are empirically fitted from baseline medians across query executions: under warm steady-state, $T_{\text{sparse}} = 79.2$ ms, $T_{\text{dense}} = 114.0$ ms, and $T_{\text{prefill}} = 423.6$ ms; under cold initialization, $T_{\text{sparse}} = 603.2$ ms, $T_{\text{dense}} = 1{,}091.7$ ms, and $T_{\text{prefill}} = 568.1$ ms. Decode

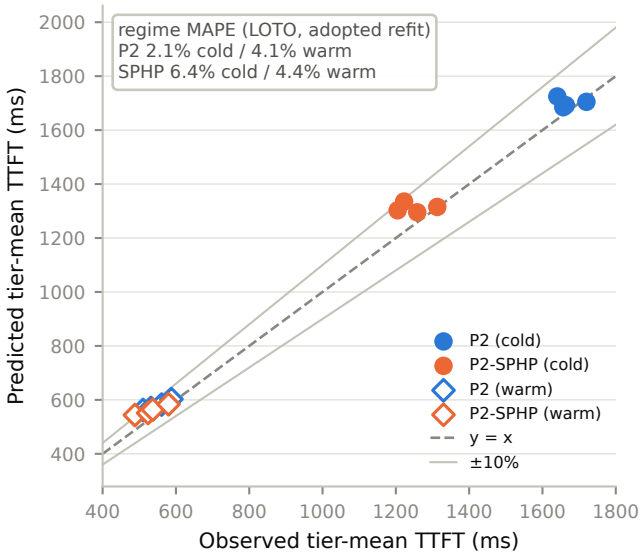


Fig. 4. Analytical cost model parity against empirical testbed measurements across cold and warm cache configurations (±10% error bounds; LOTO MAPE 2.1% cold / 4.1% warm).

TABLE III
EMPIRICAL ROBUSTNESS AND MULTI-TOPOLOGY PERFORMANCE ACROSS PHYSICAL THREE-NODE TIERING (CM-2), UNSHAPED PUBLIC WAN (CM-4), MULTI-LEG PACKET LOSS SWEEPS, AND 27-CELL STRESS MATRIX (CM-5).

Study	Nodes	Regime	P2 (ms)	SPHP	Δ	h	Loss
(a) Physical Three-Node Tiering (CM-2; $N = 240$ queries/arm)							
CM-2	3-Node	B=15, C=15	2495	2073	−16.9%	83.8%	0%
CM-2	3-Node	B=40, C=15	2784	2039	−26.7%	81.3%	0%
CM-2	3-Node	B=80, C=15	2522	2043	−19.0%	86.3%	0%
(b) Live Unshaped Public Internet Validation (CM-4; $N = 50$ queries/arm)							
CM-4	2-Node	Public Route	1731	1309	−24.4%	82.0%	live
CM-4	3-Node	Public Route	2624	1997	−23.9%	84.0%	live
(c) Link Packet Loss Sweeps & 27-Cell Stress Matrix (CM-5)							
Sweep	2-Node	RTT 0 ms, 1% loss	2649	1817	−31.4%	84.0%	1.0%
Sweep	2-Node	RTT 40 ms, 2% loss	2691	1814	−32.6%	88.0%	2.0%
CM-5	3-Node	27-Cell Mean	2688	2023	−24.7%	86.9%	0.5–2%
CM-5	Peak	B=80, loss B&C	2812	1841	−34.5%	100%	2.0%

throughput is $R_{\text{decode}} = 35.3$ tok/s. Figure 4 reports the analytical model's parity against empirical measurements in both cache states. Leave-one-tier-out cross-validation across all four evaluated network tiers (0, 15, 40, and 80 ms RTT) yields held-out mean absolute percentage errors (MAPE) of 2.15% cold / 4.13% warm for P2, and 6.42% cold / 4.37% warm for SPHP. Sweeping the validated cost model across the operating domain (RTT $\in [0, 200]$ ms, bandwidth $\in [10, 1000]$ Mbps) shows that speculative split placement (P2-SPHP) remains uniformly optimal across all 35 evaluated cells; the warm-state advantage (2.8%–3.4%) and cold-prefix advantage (23.2%) remain invariant across network RTT because identifier payloads serialize in under 0.1 ms.

V. TESTBED EVALUATION AND SENSITIVITY ANALYSIS

A. Multi-Tier WAN Evaluation and Robustness

We first evaluate the physical three-tier hierarchy enrolling Node C as edge sparse tier, Node B as neural retrieval host, and Node A as generation host across three WAN regimes ($B \in \{15, 40, 80\}$ ms, C=15 ms; $N = 240$ queries/arm; Table IIIa). Across all regimes, cold TTFT drops by 16.9%–26.7% (h = 81.3%–86.3%). Warm TTFT is nearly invariant to Link B RTT for both arms: baseline warm TTFT rises by only 9.9 ms across the 65 ms RTT span (slope 0.153 ms/ms), while SPHP warm TTFT remains flat (−11 ms over the same span).

To confirm that synthetic `netem` reflects authentic network conditions, we evaluated unshaped public internet routes (N = 50 queries/arm, `netem` disabled; Table IIIb). On the two-node route (median TCP RTT 1.10 ms, p95 4.07 ms), SPHP reduced cold TTFT from 1,730.7 ms to 1,308.6 ms (24.39% reduction, $h = 0.820$). On the three-node route originating from Node C (median RTT 4.33 ms, p95 10.46 ms), SPHP cut cold TTFT from 2,624.1 ms to 1,997.3 ms (23.89% reduction, $h = 0.840$), closely matching the synthetic emulation gains (23.7%–27.6%) under real-world jitter and residual loss.

We executed the 27-cell CM-5 stress matrix ($B \in \{15, 40, 80\}$ ms with independent per-leg losses of 0.5%, 1.0%, and 2.0% on Link B, Link C, and both simultaneously; 54 arms, 810 queries, zero transport failures; Table IIIc). SPHP achieved a mean TTFT reduction of 24.74% (peak 34.54%), maintaining a mean hit rate of 86.91%. Speedups remained uniform across tiers: 25.65% at $B = 15$ ms ($h = 85.93\%$), 22.23% at $B = 40$ ms ($h = 83.70\%$), and 26.33% at $B = 80$ ms ($h = 91.11\%$); under concurrent 2% loss on both Link B and Link C at $B = 80$ ms, SPHP achieved a 34.54% TTFT reduction with 100% hit rate.

We further assessed fault tolerance by injecting 45 s service terminations into active 365-request streaming sequences. In the two-node failure scenario (terminating Node B), exactly two requests exceeded the 30 s timeout before service restored with a Time-to-Recovery (TTR) of 86.5 s (including process re-initialization and memory warming); post-recovery latency settled within 1.4% of baseline (median 1.86 s vs. 1.83 s) with zero errors across the remaining 295 requests. In the three-node failure scenario (terminating Node C), edge failover restored service with a TTR of 64.27 s, and the first 70 post-recovery requests matched pre-fault latency (p50 2.05 s vs. 2.07 s); the restarted ingress then ceased responding, with the remaining 226 requests exceeding the 30 s bound—an instability seen only in the edge-tier failure scenario that motivates health-gated failover orchestration.

VI. DISCUSSION AND CONCLUSION

A. Discussion

Our findings contradict the intuition that WAN stage placement in distributed RAG is governed primarily by network transmission delay. When identifier-sized boundaries are maintained across the network cut, WAN RTT is a second-order factor, whereas KV-cache state swings TTFT by up to 3.2×. Distributed RAG systems engineering should therefore prioritize cache prefix warming and speculative prefill scheduling over wide-area link micro-optimizations.

Distinguishing client-observed goodput from server-internal engine throughput is critical in geo-distributed evaluation. Single-node deployments frequently rely on engine-internal

metrics (e.g., vLLM internal counters). In a distributed multi-tier architecture, internal counters track only GPU CUDA forward passes on the local instance, remaining blind to operating-system socket buffers, WireGuard encapsulation, MTU fragmentation, and HTTP SSE stream framing at the edge gateway. Measuring client-observed goodput via Equation (1) captures true end-to-end user delivery, demonstrating that decode throughput remains invariant at 35.2–35.5 tok/s across all evaluated regimes.

Speculation commitment is governed by set overlap $\text{Overlap}(\mathcal{H}, \mathcal{F}) \geq \theta$. While static $\theta = 0.5$ delivers robust acceleration ($h = 0.900$), query ambiguity benefits from adaptive calibration using BM25 score dispersion $\Delta_{\text{BM25}} = s_{\text{BM25}}(d_1) - s_{\text{BM25}}(d_{k_h})$: steep dispersion permits aggressive speculation ($\theta \to 0.4$), while flat dispersion warrants conservative verification ($\theta \to 0.7$). Our findings reflect sequential streams on MS MARCO; high-concurrency environments should combine SPHP with prefill admission control and speculation throttling. All software artifacts, traces, and scripts are open-sourced at https://github.com/ApatheticMioz/geo-distributed-hybrid-rag.

B. Conclusion

This paper presented an empirical measurement study of stage placement for hybrid RAG across geo-distributed edge–cloud tiers. We demonstrated that KV-cache state dominates latency variance (2.8–3.2×) while TTFT is flat across WAN delays (≈0.95 ms/ms) due to identifier-sized boundaries. Exploiting this retrieval-prefill asymmetry, SPHP overlaps masked GPU prefill with in-flight neural retrieval, cutting cold TTFT by 16.9%–26.7% on the physical three-node testbed without factual degradation while warm Link B sensitivity stays at or below 0.153 ms/ms. Our analytical cost model establishes the advantage of speculative split placement across the swept operating domain, corroborated by multi-tier physical experiments (CM-2), unshaped public WAN routes (CM-4), and multi-leg stress topologies (CM-5).

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9781–9793.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *International Conference on Machine Learning (ICML)*, 2020, pp. 3979–3988.

[3] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, A. Lucier, F. Hutter *et al.*, "Improving language models by retrieving from trillions of tokens," in *International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.

[4] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2024, arXiv:2312.10997.

[5] S. Robertson and S. Walker, "Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval," in *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1994, pp. 232–241.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[7] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[8] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 135–142.

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[10] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629.

[11] A. Banitalebi-Dehkordi, N. Vedula, J. Pei, F. Xia, L. Wang, and Y. Zhang, "Auto-Split: A general framework of collaborative edge-cloud AI," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining (KDD)*, 2021, pp. 2543–2553.

[12] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 1–11.

[13] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

[14] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 193–210.

[15] K. B. Aruna, V. Kaliraj, I. Sudha, and V. Sureka, "Janus: Joint prefill/decode disaggregation with KV-cache-aware multi-cloud routing for edge-adjacent LLM serving," *EAI Endorsed Transactions on the Internet of Things*, 2026.

[16] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, pp. 155–170, best Paper.

[17] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, and R. Vinayak, "Helix: Serving large language models over heterogeneous GPUs and network via max-flow," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025, pp. 586–602.

[18] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, and T. Wang, "MS MARCO: A human generated machine reading comprehension dataset," 2016, arXiv:1611.09268, v3 2018.

[19] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, "Towards understanding systems trade-offs in retrieval-augmented generation model inference," 2024, arXiv:2412.11854.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202*, 2023, pp. 19 274–19 286.

[21] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J.-J. Cappe, "Accelerating large language model decoding with speculative sampling," 2023, arXiv:2302.01318.

[22] Y. Lyu, B. Irukulapati, and J. P. Champati, "Speculation at a distance: Where edge–cloud speculative decoding actually pays off," 2026, arXiv:2606.25091.

[23] Z. Wang, Z. Wang, L. Le, H. S. Zheng, S. Mishra, V. Perot, Y. Zhang, A. Mattapalli, A. Taly, J. Shang, C. Lee, and T. Pfister, "Speculative RAG: Enhancing retrieval-augmented generation through drafting," in *International Conference on Learning Representations (ICLR)*, 2025.

[24] R. Fu, S. Li, Y. Xu, T. Wu, Z. Ma, and J. Shi, "Omnia: Efficient RAG serving through speculative scheduling," in *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '26)*, 2026.

[25] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[26] Qwen Team, "Qwen3.8-27b (hugging face model card)," https://huggingface.co/Qwen/Qwen3.8-27B, 2026, qwen3.8-27B release; W4A16 AutoRound-quantized checkpoint https://huggingface.co/born2bewild/Qwen3.8-27B-W4A16-AutoRound-fast.

[27] W. Cheng, W. Zhang, H. Shen, Y. Cai, X. Y. He, K. Lv, and Y. Liu, "Optimize weight rounding via signed gradient descent for the quantization of LLMs," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, autoRound / SignRound; INT4 (W4A16) via compressed-tensors.

[28] iproute2 project, "netem(8): Network emulator (tc qdisc)," iproute2 manual page, 2024, [Online]. Available: https://man7.org/linux/man-pages/man8/tc-netem.8.html.